Defiende el servidor: versionado y recuperación de un detector de ataques

MLOps · Visualización 2 · 2026-II · Grupo B · Rafael Orozco (individual) · Universidad Externado de Colombia · 31 de octubre de 2026

1. Objetivo y trazabilidad

Se construyó un clasificador binario normal (0) / ataque (1, clase positiva) con las particiones publicadas de UNSW-NB15. El foco es que cada versión pueda identificarse y recuperarse: el código vive en Git/GitHub, los datos en DVC (remote HTTP en un repo dataset público de Hugging Face), los experimentos en MLflow (exportados a evidencias/mlflow) y los modelos en Hugging Face, referenciados por SHA completo. Cada pieza guarda el identificador de la anterior.

	v1	v2
Configuración / umbral	E1-logreg-base / 0.5	E4-hgb / 0.64
Commit Git que entrenó	d7011d2272daf4fab8555744bbcaeebe49f5cf08	cd74ed860fc4c812e886272a6de16a82809ecfc9
Datos DVC (crudos · preparados)	fe93051ae209ffb0128a31e7a478c009.dir · 731ffba210f38c4b5fed86beb0d6eff7.dir	iguales a v1
Run ID MLflow	8a9c17ff6fb94677aee381b05b8e4827	6bb2e44a2de843f8ac5dfff5c77cc865
Revisión HF (Rafoniloni/defiende-el-servidor-unsw)	264e987803bcae3aaa09c719df77fe8d726eb96d	64fc9774c0779005d65f79c769a4def98258065a
sha256 model.joblib	68b0defa2b8ad78e682489b3cadb03fd04368c06accacaf3fcd544d864a5cdfa	b32254d6bf876d341287690214a015e463a9186e3186b8e28d4c5df12310ce1b

2. Datos, auditoría y limpieza

Se usaron UNSW_NB15_training-set.csv (175.341 filas) y UNSW_NB15_testing-set.csv (82.332 filas), descargados el 2026-10-31 de un mirror fijado por commit. Sus SHA-256 coinciden con el anexo de la guía, y también las filas y el esquema (45 columnas). No hay valores faltantes; en service, '-' es una categoría. Las variables numéricas son ≥ 0. Se excluyen id (identificador; se conserva como row_id solo para auditoría), attack_cat (revela la respuesta) y label, de modo que quedan 42 predictoras (3 categóricas).

Duplicados por entradas. En el archivo de entrenamiento hay 101.040 combinaciones únicas de predictoras; 9.748 grupos se repiten y suman 74.301 filas extra. 229 grupos (940 filas) tienen las mismas entradas con etiquetas distintas. En test: 28.386 filas extra y 6 grupos contradictorios. **Política:** los grupos contradictorios se eliminan solo de train/val, porque no son aprendibles. Los demás duplicados se conservan como frecuencia observada, y cada grupo va completo a un solo split. El test no se toca.

Particiones. Validación = grupos con sha256(semilla|grupo) mod 10 000 < 2 000 (semilla 42). La asignación es determinista y no depende de la versión de la librería; queda guardada fila a fila en asignacion.csv. Train: 139.756 filas (95.268 ataques); validación: 34.645 (23.529 ataques); test: 82.332 (45.332 ataques). Grupos compartidos entre train y val: **0**. El 10,1 % de las filas de test (8.313) repite entradas de train/val, y 451 de ellas con otra etiqueta. Codificadores, escalado y modelo se ajustan solo con train dentro de un único Pipeline de scikit-learn. Los CSV recibidos (data/raw.dvc) y los preparados (dvc.lock) son las dos versiones de datos en DVC.

3. Cuatro configuraciones en dos familias

Todas usan la misma partición, la semilla 42 y umbral 0,5 para compararse. Cada run registra en MLflow la hipótesis, el cambio, los parámetros, el commit, los md5 de DVC, las métricas de validación, el pipeline completo y su sha256. Las métricas toman ataque como positivo: FPR = FP/(FP+TN). Si un denominador es 0, la métrica se reporta como null y aporta 0 al macro-F1; no ocurrió en ningún run. Validación: n = 34.645.

ID / familia	Hipótesis y cambio	Recall	Prec.	FPR	Macro-F1	FP / FN	Run ID
E1-logreg-base lineal	Referencia sencilla (sin cambio previo). Una regresión logística con one-hot y escalado separa buena parte del tráfico, pero las variables de conteo/bytes con colas pesadas limitarán su frontera lineal.	0,988	0,927	0,164	0,927	1.822 / 292	8a9c17ff6f…
E2-logreg-log1p lineal	Respecto a E1, se aplica log1p a las variables numéricas (único cambio). Comprimir las colas pesadas (bytes, cargas, tiempos) con log1p hará más lineales las relaciones y reducirá falsos positivos frente a E1 con el mismo estimador.	0,975	0,942	0,126	0,933	1.404 / 578	420c54b7aa…
E3-rf arboles	Cambio de familia a ensamble de árboles (bagging). Un bosque aleatorio captura interacciones no lineales (p. ej. ttl x estado) y superará a la familia lineal en macro-F1 sin necesidad de transformar escalas.	0,978	0,961	0,085	0,951	945 / 515	5d3bda042f…

ID / familia	Hipótesis y cambio	Recall	Prec.	FPR	Macro-F1	FP / FN	Run ID
E4-hgb arboles	Respecto a E3, boosting secuencial en lugar de bagging. El boosting por histogramas corrige errores residuales del bosque y logra un macro-F1 igual o mayor con un artefacto mucho más pequeño.	0,976	0,965	0,074	0,953	822 / 574	6bb2e44a2d...

Interpretación. Las cuatro hipótesis se confirmaron. En la familia lineal, log1p reduce las falsas alarmas un 23 % (1.822 → 1.404), pero pierde recall: la frontera lineal no separa bien el tráfico normal voluminoso. Los árboles capturan interacciones (TTL × estado, bytes × servicio) y reducen la FPR a la mitad. E4 (boosting) iguala o supera a E3 (bagging) con un artefacto 18 veces menor (1 MB frente a 17 MB). En todos los modelos, el error dominante son conexiones normales marcadas como ataque, no ataques perdidos: el ataque es la clase mayoritaria (68 %).

4. De v1 a v2 y elección del umbral

v1 es la primera solución funcional: E1 con umbral 0,5. **v2** sale de una regla declarada en configs/experimentos.yaml antes de entrenar E2–E4 y aplicada solo sobre validación: (1) elegir la configuración con mayor macro-F1 a 0,5, que es E4-hgb; (2) elegir el umbral con mayor recall de ataque sujeto a FPR ≤ 0,05, en una rejilla de 0,01 a 0,99 con paso 0,01. A 0,5, E4 tenía FPR 0,074, por encima del límite; el primer umbral que lo cumple es **0,64**. Para un equipo de seguridad, una FPR de 0,164 (1 de cada 6 conexiones normales alertada) satura la operación; por eso se priorizó acotarla.

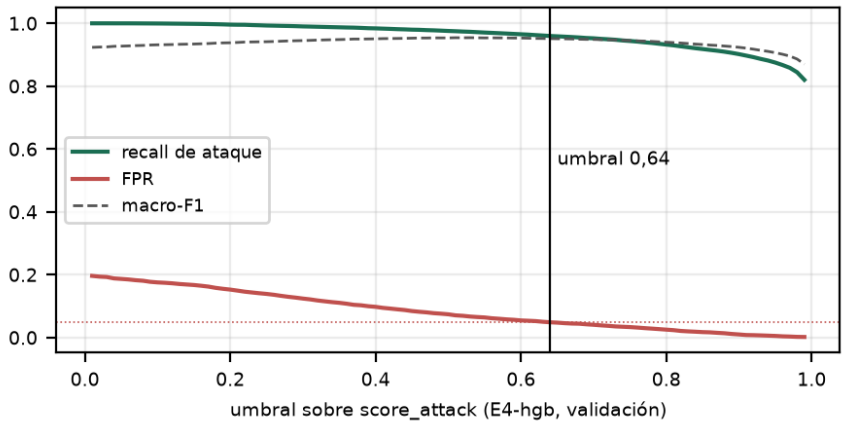


Figura 1. Curva de umbral de E4 en validación (evidencias/umbral_v2.csv). Línea punteada: FPR máxima; vertical: umbral elegido.

Validación	Recall	Precision	FPR	Macro-F1	TN	FP	FN	TP
v1 (E1, 0,5)	0,988	0,927	0,164	0,927	9.294	1.822	292	23.237
v2 (E4, 0,64)	0,960	0,977	0,049	0,951	10.576	540	948	22.581

Qué mejora: la FPR baja 0,115 (las falsas alarmas pasan de 1.822 a 540), la precisión sube 0,049 y el macro-F1 0,024. **Qué empeora:** el recall baja 0,028 y los ataques no detectados pasan de 292 a 948. **Error aceptado:** perder unos 3 de cada 100 ataques a cambio de unas 3,4 veces menos alertas falsas. Como el umbral se eligió sobre la misma validación, estas cifras de v2 son algo optimistas. La comparación v1/v2 se hace solo en validación.

5. Evaluación final en test

La selección se congeló en el commit 2f82c1bbf6... (2026-10-31T08:50:13-05:00) con la revisión de Hugging Face ya fijada. Después, evaluar_test.py descargó v2 por SHA (64fc9774c0...), verificó el sha256 del modelo y predijo una sola vez las 82.332 filas del test, sin eliminar ninguna. El script se niega a re-evaluar. predicciones-test.csv tiene una fila por registro (row_id = testing-set:<fila>, y_true, score_attack, y_pred = score ≥ 0,64); las métricas se recalcularon desde ese archivo y coinciden.

Test (v2)	Recall	Precision	FPR	Macro-F1	Accuracy	TN	FP	FN	TP
n = 82.332	0,971	0,854	0,203	0,890	0,893	29.498	7.502	1.307	44.025

El resultado empeora y se conserva. El recall se mantiene (0,971), pero la FPR sube de 0,049 a 0,203. Un análisis posterior (diagnostico-test.json, que no cambia nada de lo decidido) muestra que las conexiones normales del archivo de test reciben puntuaciones mucho más altas (Figura 2). Por estado de conexión, la FPR pasa de 0,054 a 0,202 en FIN y de 0,133 a 0,578 en INT. No se explica por el solapamiento: las filas de test no vistas en train/val tienen una FPR de 0,202, y las vistas, de 0,226. Es un cambio de distribución entre las particiones publicadas: la validación, tomada del mismo archivo que train, no lo anticipaba. Ajustar el umbral con el test lo invalidaría como evaluación final, así que no se hizo.

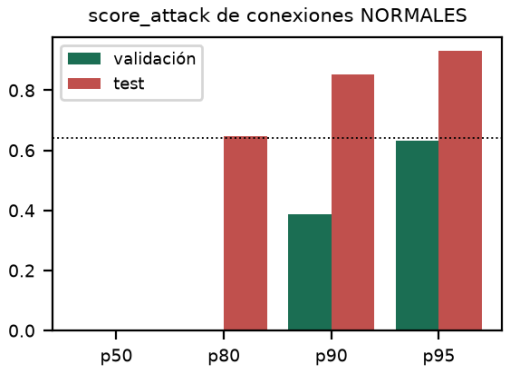


Figura 2. Cuantiles del score de las conexiones normales; la línea punteada es el umbral 0,64.

6. Prueba de recuperación

scripts/prueba_recuperacion.sh clonó GitHub en una carpeta temporal nueva, con cachés de uv, DVC y Hugging Face vacías. Instaló las dependencias fijadas (uv sync --frozen), ejecutó dvc pull desde el remote público y verificó los 6 SHA-256 de los datos crudos y preparados. Luego descargó v1 y v2 por su SHA completo, sin credenciales, y predijo las mismas 25 filas fijas de validación. Diferencia máxima de score: v1 = 0.0, v2 = 0.0 (tolerancia 1e-09); etiquetas iguales y CSV con hashes idénticos a los de referencia. El log y las salidas están en evidencias/recuperacion/prueba.

7. Limitaciones

(i) UNSW-NB15 es tráfico de laboratorio de 2015 y no demuestra eficacia en una red actual. (ii) Hay un cambio de distribución entre train y test: la FPR operativa real sería mayor que la estimada en validación, y una validación del mismo archivo resulta optimista. (iii) El umbral de v2 se eligió y reportó sobre la misma validación. (iv) Los duplicados se conservan y pesan en las métricas; el 10 % del test repite entradas vistas. (v) El modelo es un pickle de scikit-learn 1.9.1 y solo debe cargarse con esas versiones y desde fuentes confiables. (vi) MLflow corrió en local; la evidencia consultable es la exportación en JSON/CSV.

Referencias

Moustafa, N. y Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS 2015, IEEE. https://research.unsw.edu.au/projects/unsw-nb15-dataset · Documentación de Git, DVC, MLflow Tracking, Hugging Face Hub (descarga por revisión, model cards) y scikit-learn (common pitfalls). Uso de IA: Claude Code (Claude Opus 5.5) apoyó el código y la redacción; ver entrega.md.